

**Sri Sivasubramaniya Nadar College of Engineering, Chennai**  
(An autonomous Institution affiliated to Anna University)

|                     |                                                    |                  |                         |
|---------------------|----------------------------------------------------|------------------|-------------------------|
| Degree & Branch     | M.Tech (Integrated) Computer Science & Engineering | Semester         | V                       |
| Subject Code & Name | ICS1512 - Machine Learning Algorithms Laboratory   |                  |                         |
| Academic year       | 2025-2026 (Odd)                                    | Batch: 2023-2028 | <b>Date: 02/09/2025</b> |

## Experiment 4 : Ensemble Prediction and Decision Tree Model Evaluation

### 1 Aim and Objective

The objective of this experiment is to build and evaluate various classification models on the Wisconsin Diagnostic Dataset. This includes a single Decision Tree classifier and several ensemble methods such as AdaBoost, Gradient Boosting, XGBoost, Random Forest, and a Stacking Classifier. The performance of these models is rigorously assessed using 5-Fold Cross-Validation and hyperparameter tuning to identify the most effective model for this dataset.

### 2 Libraries Used

The primary libraries used for this experiment were:

- **pandas** and **numpy** for data manipulation and numerical operations.
- **matplotlib** and **seaborn** for data visualization.
- **scikit-learn** for implementing models, preprocessing data, and evaluating performance.
- **xgboost** for the implementation of the XGBoost classifier.
- **google.colab** for mounting Google Drive to access the dataset.

### 3 Data Load And Preprocessing

```
column_names = [  
    'ID', 'Diagnosis',  
    'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean',
```

```

        'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se', 'compactness_se',
        'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst', 'smoothness_worst',
    ]

data_path = "/content/drive/MyDrive/ML_LAB/BreastCancer_WinconsinDiagnostic/wdbc.data"

df = pd.read_csv(data_path, header=None, names=column_names)

print("Shape of raw dataset:", df.shape)

df.head()

```

Shape of raw dataset: (569, 32)

|   | ID       | Diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | smoothness_mean | compactness_mean | concavity_mean | concave points_mean | ... | radius_worst | texture_worst |
|---|----------|-----------|-------------|--------------|----------------|-----------|-----------------|------------------|----------------|---------------------|-----|--------------|---------------|
| 0 | 842302   | M         | 17.99       | 10.38        | 122.80         | 1001.0    | 0.11840         | 0.27760          | 0.3001         | 0.14710             | ... | 25.38        | 17.99         |
| 1 | 842517   | M         | 20.57       | 17.77        | 132.90         | 1326.0    | 0.08474         | 0.07864          | 0.0869         | 0.07017             | ... | 24.99        | 17.77         |
| 2 | 84300903 | M         | 19.69       | 21.25        | 130.00         | 1203.0    | 0.10960         | 0.15990          | 0.1974         | 0.12790             | ... | 23.57        | 21.25         |
| 3 | 84348301 | M         | 11.42       | 20.38        | 77.58          | 386.1     | 0.14250         | 0.28390          | 0.2414         | 0.10520             | ... | 14.91        | 20.38         |
| 4 | 84358402 | M         | 20.29       | 14.34        | 135.10         | 1297.0    | 0.10030         | 0.13280          | 0.1980         | 0.10430             | ... | 22.54        | 14.34         |

5 rows x 32 columns

Figure 1: Output , showing the first five records of the dataset.

```

# Drop the ID column as it is not a predictive feature
df_processed = df.drop(columns=['ID'])

# Define the target column for convenience
TARGET_COLUMN = 'Diagnosis'

# This keeps the original 'M' and 'B' labels for plotting.
df_eda = df_processed.copy()

# Encode the target variable: 'M' (malignant) and 'B' (benign) will be converted to 1 and 0
le = LabelEncoder()
df_processed[TARGET_COLUMN] = le.fit_transform(df_processed[TARGET_COLUMN])

# Separate features (X) and the target variable (y) from the processed DataFrame
X = df_processed.drop(columns=[TARGET_COLUMN])
y = df_processed[TARGET_COLUMN]

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

## 4 EDA

### Class Distribution

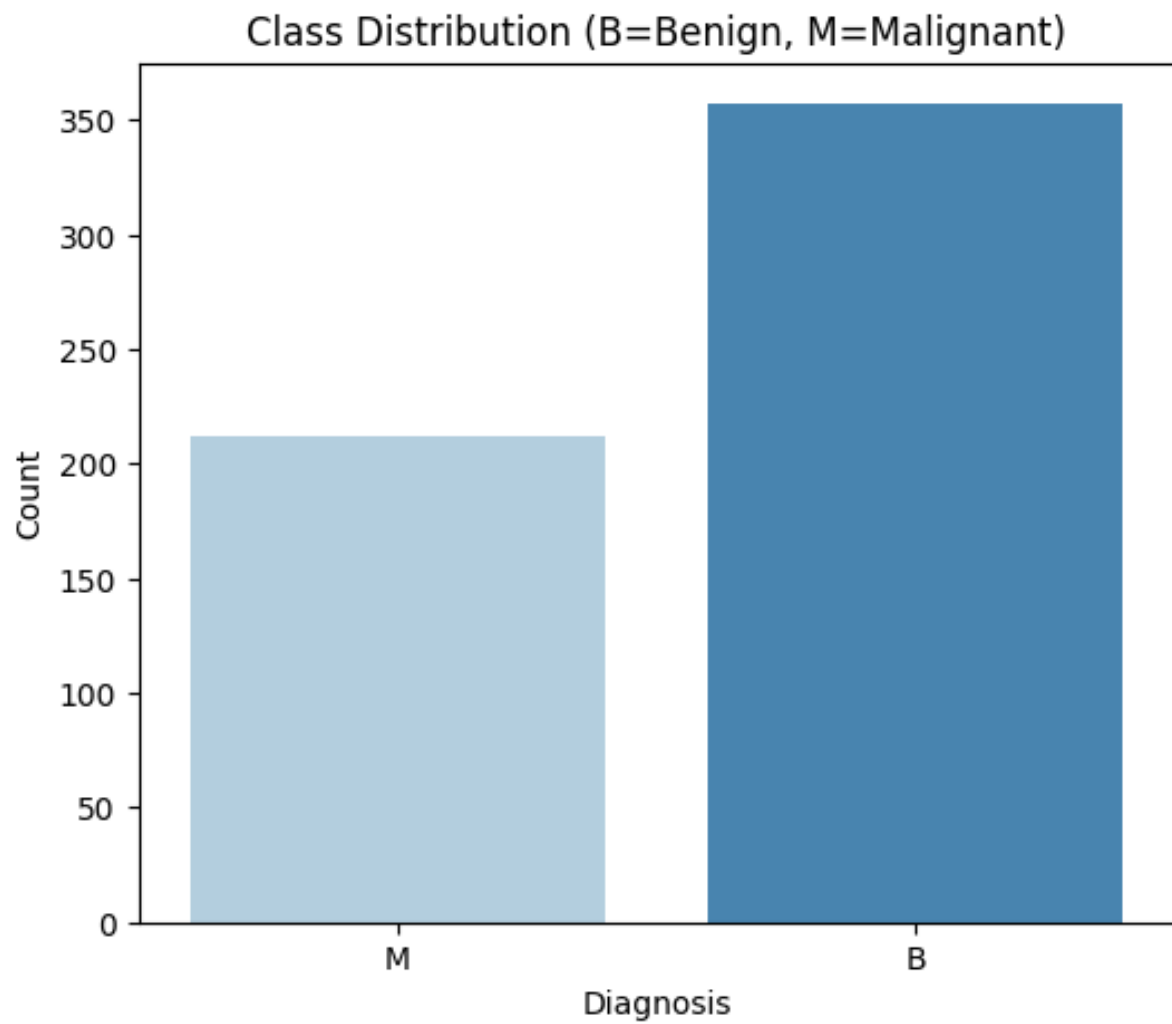


Figure 2: Distribution of Benign vs. Malignant cases in the dataset.

## Feature Distributions (Histograms)

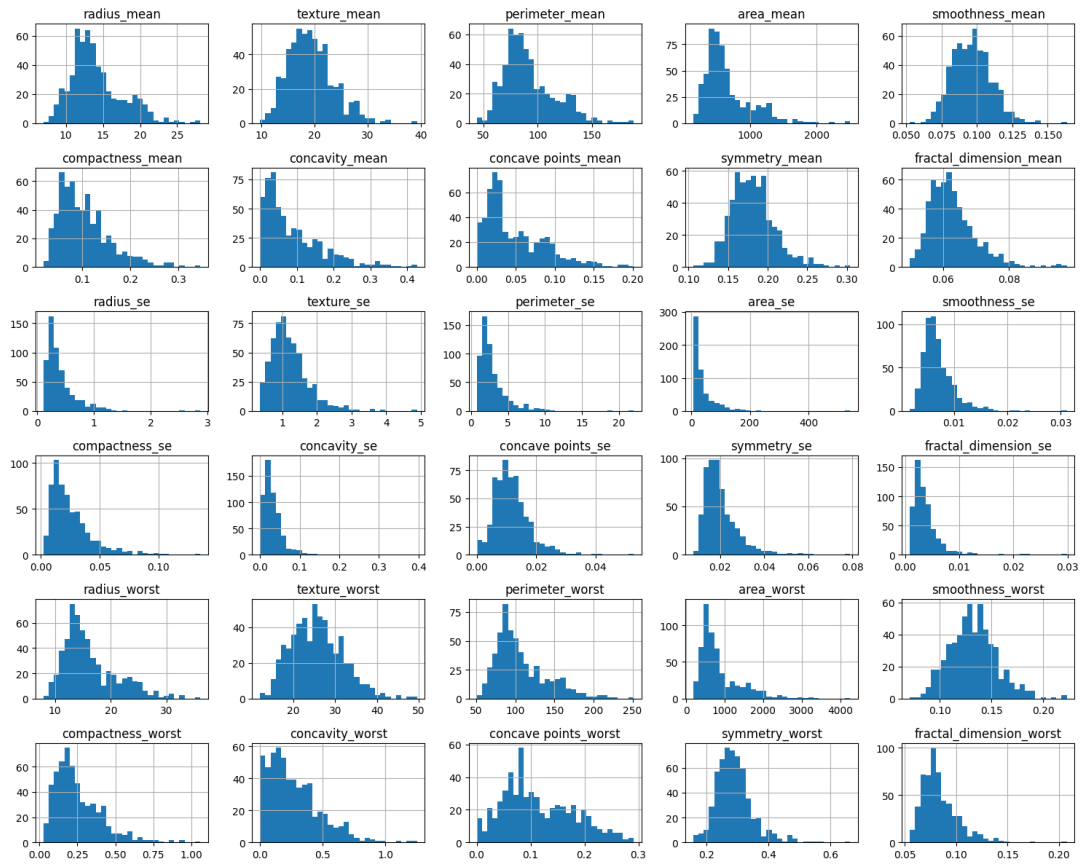


Figure 3: Distribution of each of the 30 numerical features in the Wisconsin Breast Cancer dataset.

## Correlation Heatmap

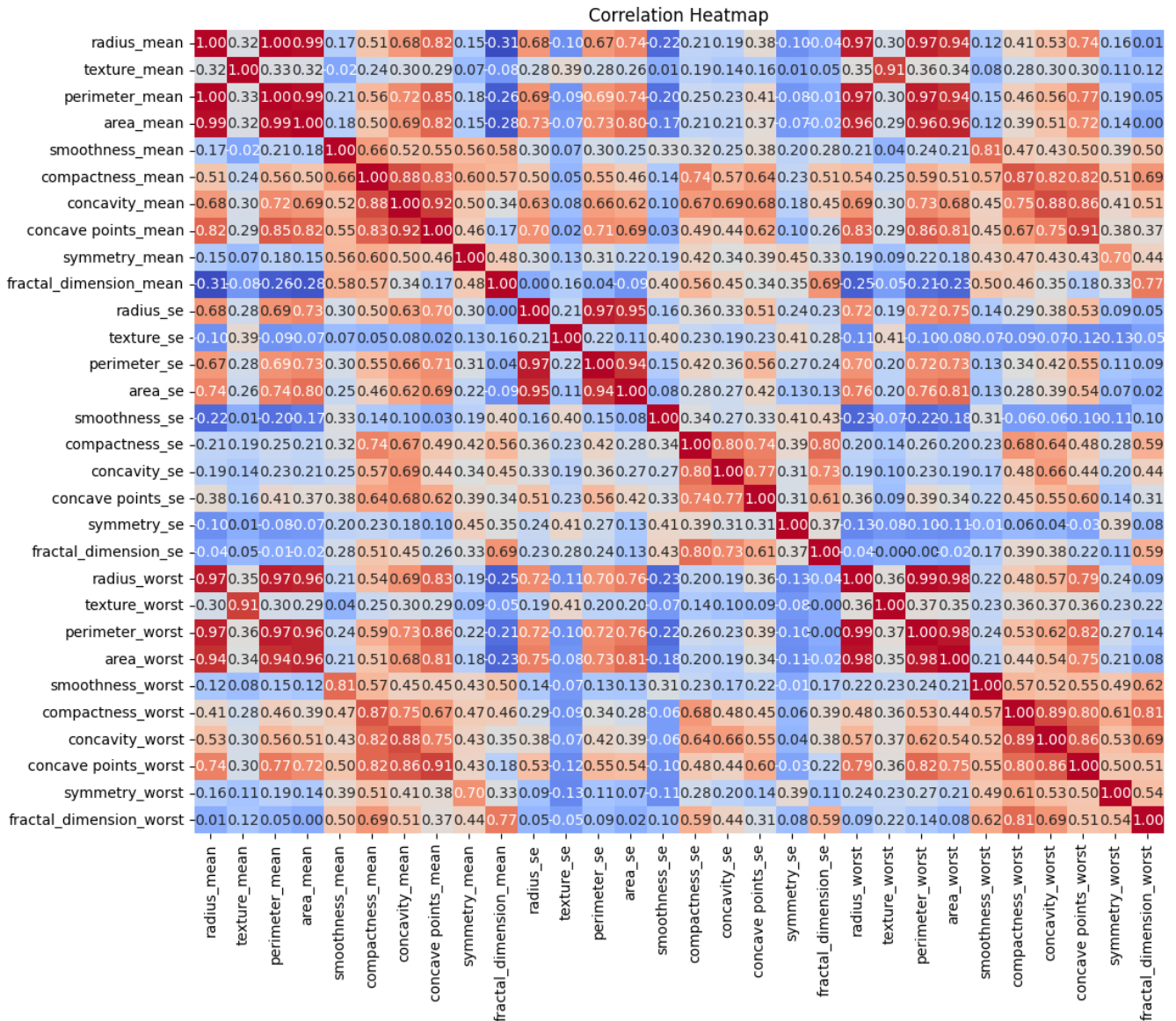


Figure 4: Heatmap illustrates the Pearson correlation between all features.

## 5 Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled,
y, test_size=0.2, random_state=42, stratify=y)
print("\nPreprocessing and Splitting Complete")
print(f"Training set shape: {X_train.shape}")
print(f"Test set shape: {X_test.shape}\n")
```

```
Preprocessing and Splitting Complete
Training set shape: (455, 30)
Test set shape: (114, 30)
```

## 6 Model Training and Evaluation

### Model training and evaluation function

```
cv_results = {} # This should be defined before the function

def train_and_evaluate_model(model, param_grid, model_name, X_train, y_train, X_test, ):

    print(f" Training and Evaluating: {model_name} ")

    # MODIFICATION: Define multiple scoring metrics and tell GridSearchCV to refit by
    scoring_metrics = ['accuracy', 'f1_weighted']
    grid_search = GridSearchCV(
        estimator=model,
        param_grid=param_grid,
        cv=5,
        scoring=scoring_metrics,
        refit='accuracy', # The model will be refit on the best 'accuracy' score
        n_jobs=-1
    )
    grid_search.fit(X_train, y_train)

    best_model = grid_search.best_estimator_
    print(f" Best Hyperparameters found: {grid_search.best_params_}")

    print("\n Top Hyperparameter Trials (sorted by Accuracy) ")
    results_df = pd.DataFrame(grid_search.cv_results_)

    # Select only the columns we need for the table
    param_cols = [col for col in results_df.columns if col.startswith('param_')]
    metric_cols = ['mean_test_accuracy', 'mean_test_f1_weighted']

    # Create a cleaner table for display
    trials_table = results_df[param_cols + metric_cols].copy()

    # Clean up column names
    trials_table.columns = [c.replace('param_', '') for c in trials_table.columns]
    trials_table.rename(columns={'mean_test_accuracy': 'Accuracy', 'mean_test_f1_weighted': 'F1 Score'})

    # Print the top 10 best performing combinations
    print(trials_table.sort_values(by='Accuracy', ascending=False).round(4).head(10))

    print(f"\n Best 5-Fold CV Accuracy (from the search): \

    {grid_search.best_score_:.4f}")
```

```

# Store the cross-validation scores for the final summary table
cv_scores = cross_val_score(best_model, X_train, y_train, cv=5, scoring='accuracy')
cv_results[model_name] = cv_scores

# Evaluate the final model on the test set
y_pred = best_model.predict(X_test)
print(f"Final Test Set Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")

# Plotting
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle(f'Evaluation Metrics for {model_name}', fontsize=16)
ConfusionMatrixDisplay.from_estimator(best_model, X_test, y_test, cmap='Blues', ax=axes[0])
axes[0].set_title('Confusion Matrix')
RocCurveDisplay.from_estimator(best_model, X_test, y_test, ax=axes[1])
axes[1].set_title('ROC Curve')
plt.show()

if hasattr(best_model, 'feature_importances_'):
    feature_imp = pd.Series(best_model.feature_importances_, index=feature_names)
    plt.figure(figsize=(10, 6))
    sns.barplot(x=feature_imp, y=feature_imp.index)
    plt.title(f"Top 10 Feature Importances for {model_name}")
    plt.xlabel("Feature Importance Score")
    plt.ylabel("Features")
    plt.show()

```

## Decision Tree

```

dt_model = DecisionTreeClassifier(random_state=42)
dt_param_grid = {'criterion': ['gini', 'entropy'], 'max_depth': [3, 5, 7, 10], 'min_samples_split': [2, 5, 10]}
train_and_evaluate_model(dt_model, dt_param_grid, 'Decision Tree', X_train, y_train, cv_results)

```

Best 5-Fold CV Accuracy (from the search): 0.9451

Final Test Set Accuracy: 0.9474

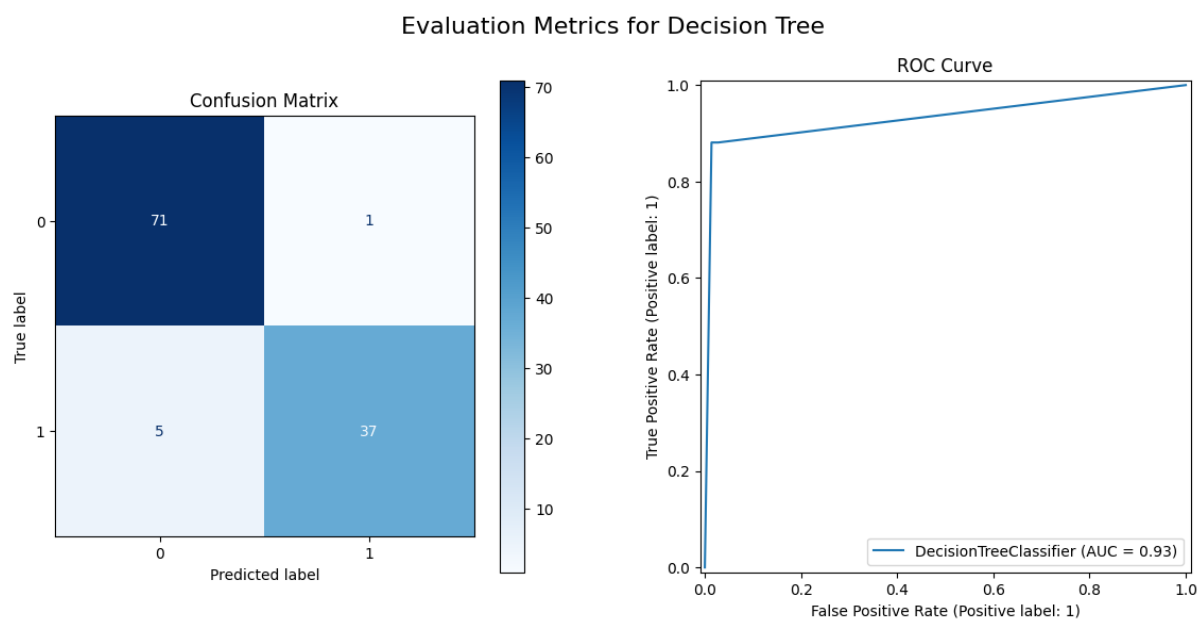


Figure 5: Confusion Matrix and ROC Curve for Decision Tree model.

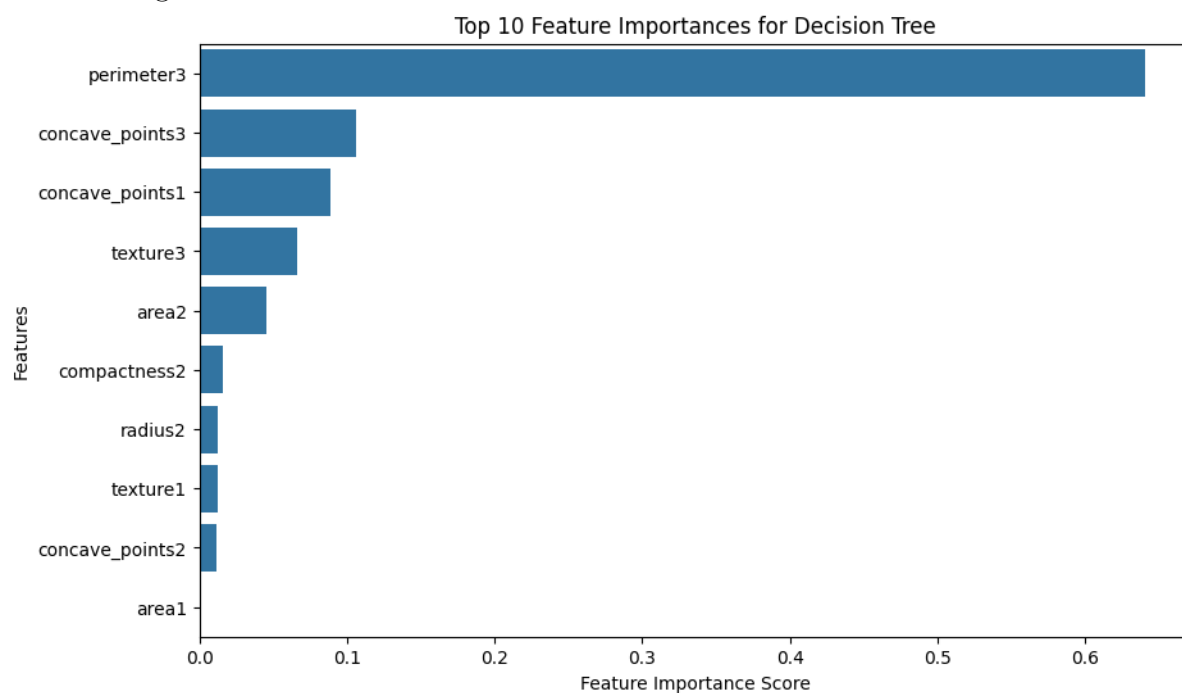


Figure 6: Feature importance scores for the tuned Decision Tree model.



## Hyperparameter Tuning

Table 1: Decision Tree Hyperparameter Tuning Trials

| <b>criterion</b> | <b>max_depth</b> | <b>Accuracy</b> | <b>F1_score</b> |
|------------------|------------------|-----------------|-----------------|
| entropy          | 7                | 0.9451          | 0.9452          |
| entropy          | 10               | 0.9451          | 0.9452          |
| gini             | 7                | 0.9429          | 0.9427          |
| gini             | 10               | 0.9385          | 0.9383          |
| entropy          | 10               | 0.9363          | 0.9365          |
| entropy          | 7                | 0.9363          | 0.9365          |
| entropy          | 10               | 0.9341          | 0.9342          |
| entropy          | 5                | 0.9341          | 0.9341          |
| entropy          | 7                | 0.9341          | 0.9342          |
| entropy          | 5                | 0.9341          | 0.9343          |

## AdaBoost

```
ada_model = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), random_
ada_param_grid = {'n_estimators': [50, 100, 200], 'learning_rate': [0.01, 0.1, 1.0]}z
train_and_evaluate_model(AdaBoostClassifier(estimator=DecisionTreeClassifier(max_dept
```

B Best 5-Fold CV Accuracy (from the search): 0.9648

Final Test Set Accuracy: 0.9737

## Hyperparameter Tuning

Table 2: AdaBoost Hyperparameter Tuning Trials

| <b>learning_rate</b> | <b>n_estimators</b> | <b>Accuracy</b> | <b>F1_score</b> |
|----------------------|---------------------|-----------------|-----------------|
| 1.00                 | 200                 | 0.9648          | 0.9648          |
| 1.00                 | 50                  | 0.9626          | 0.9625          |
| 1.00                 | 100                 | 0.9604          | 0.9603          |
| 0.10                 | 200                 | 0.9516          | 0.9516          |
| 0.10                 | 100                 | 0.9516          | 0.9517          |
| 0.10                 | 50                  | 0.9495          | 0.9493          |
| 0.01                 | 200                 | 0.9319          | 0.9313          |
| 0.01                 | 100                 | 0.9253          | 0.9245          |
| 0.01                 | 50                  | 0.9231          | 0.9220          |

## Gradient Boost

```
gb_model = GradientBoostingClassifier(random_state=42)
gb_param_grid = {'n_estimators': [100, 200], 'learning_rate': [0.05, 0.1], 'max_depth
train_and_evaluate_model(gb_model, gb_param_grid, 'Gradient Boosting', X_train, y_tra
```

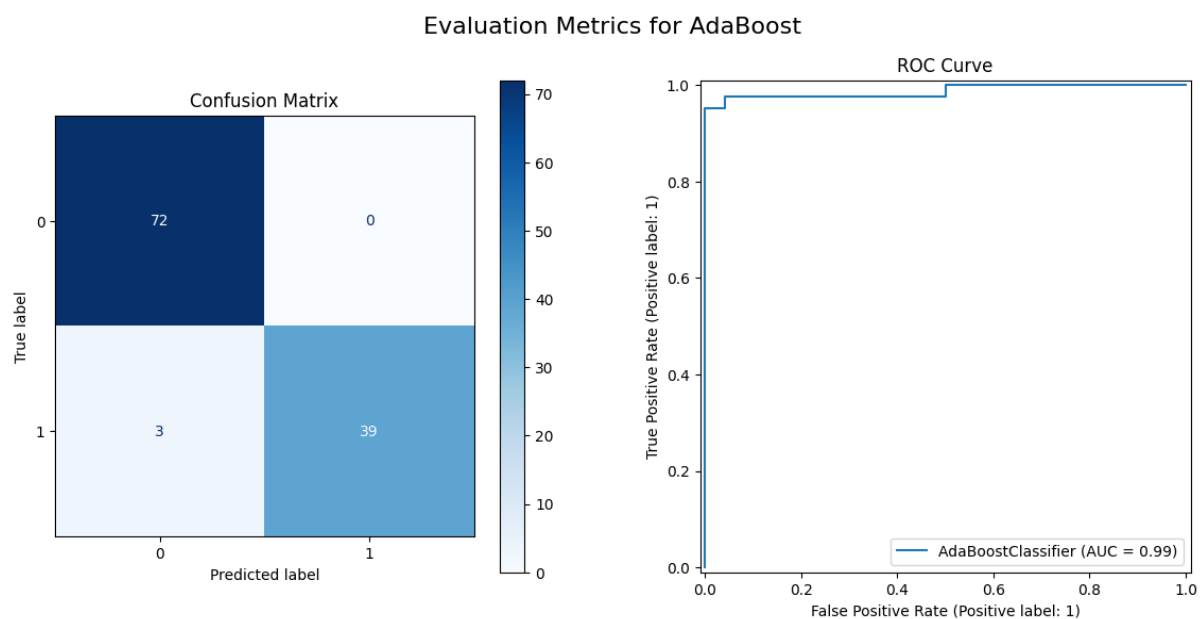


Figure 7: Confusion Matrix and ROC Curve for AdaBoost model.

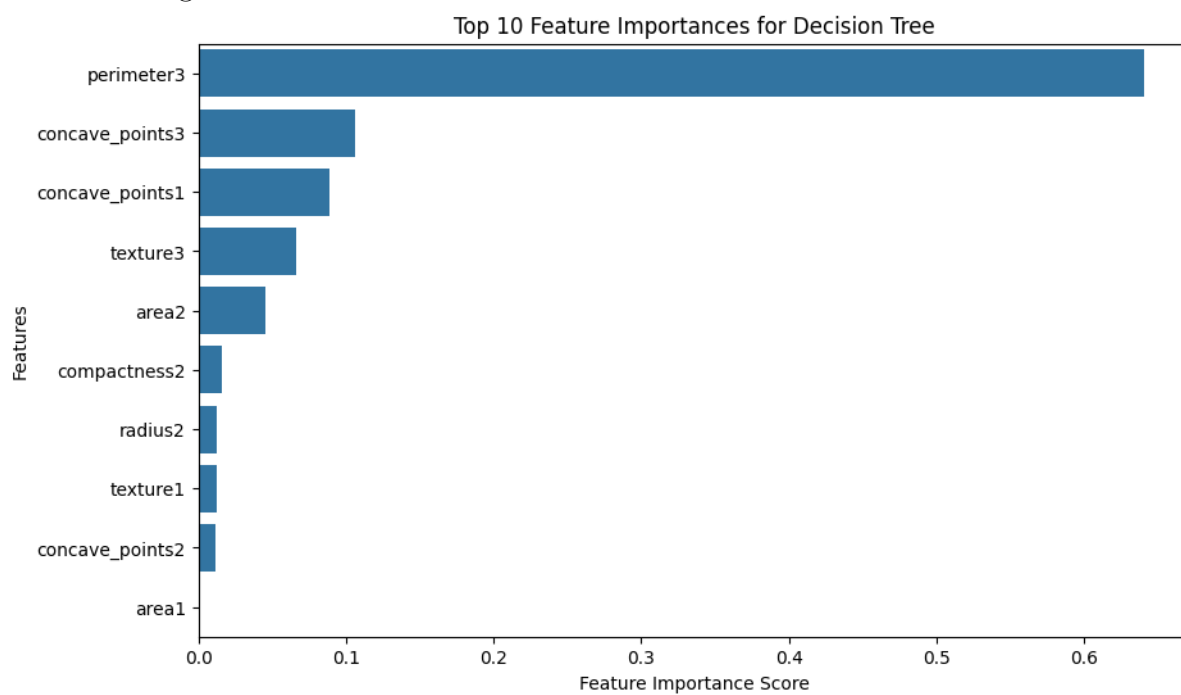


Figure 8: Feature importance scores for the tuned AdaBoost model.

Best 5-Fold CV Accuracy (from the search): 0.9538  
 Final Test Set Accuracy: 0.9649

## Hyperparameter Tuning

Table 3: Gradient Boosting Hyperparameter Tuning Trials

| learning_rate | max_depth | n_estimators | Accuracy | F1_score |
|---------------|-----------|--------------|----------|----------|
| 0.10          | 3         | 200          | 0.9538   | 0.9538   |
| 0.10          | 3         | 100          | 0.9538   | 0.9538   |
| 0.05          | 3         | 100          | 0.9516   | 0.9515   |
| 0.05          | 3         | 200          | 0.9516   | 0.9516   |
| 0.05          | 5         | 100          | 0.9407   | 0.9405   |
| 0.10          | 5         | 100          | 0.9297   | 0.9299   |
| 0.05          | 5         | 200          | 0.9275   | 0.9275   |
| 0.10          | 5         | 200          | 0.9209   | 0.9212   |

## XG Boost

```
xgb_model = XGBClassifier(random_state=42, eval_metric='logloss')
xgb_param_grid = {'n_estimators': [100, 200], 'learning_rate': [0.05, 0.1], 'max_depth': [3, 5]}
train_and_evaluate_model(xgb_model, xgb_param_grid, 'XGBoost', X_train, y_train, X_test, y_test)
```

Best 5-Fold CV Accuracy (from the search): 0.9648  
 Final Test Set Accuracy: 0.9649

## Hyperparameter Tuning

Table 4: XGBoost Hyperparameter Tuning Trials

| learning_rate | max_depth | n_estimators | Accuracy | F1_score |
|---------------|-----------|--------------|----------|----------|
| 0.05          | 5         | 200          | 0.9648   | 0.9649   |
| 0.10          | 5         | 200          | 0.9626   | 0.9627   |
| 0.10          | 5         | 100          | 0.9626   | 0.9627   |
| 0.05          | 3         | 200          | 0.9582   | 0.9582   |
| 0.10          | 3         | 200          | 0.9582   | 0.9582   |
| 0.10          | 3         | 100          | 0.9582   | 0.9583   |
| 0.05          | 5         | 100          | 0.9538   | 0.9539   |
| 0.05          | 3         | 100          | 0.9538   | 0.9540   |

## Random Forest

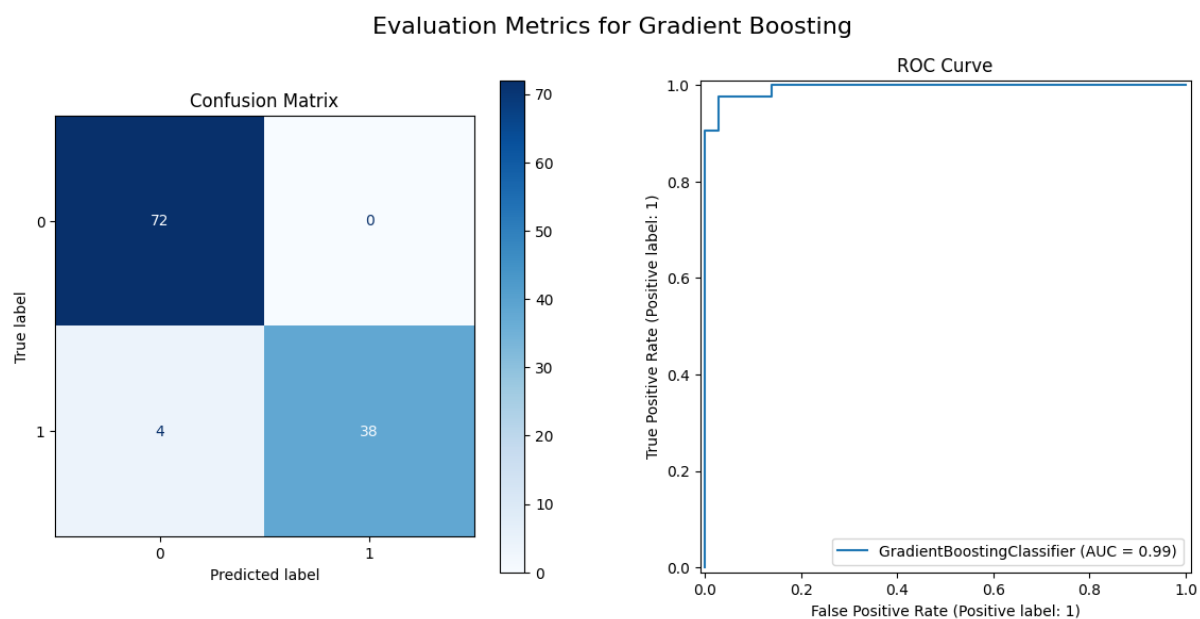


Figure 9: Confusion Matrix and ROC Curve for Gradient Boost model.

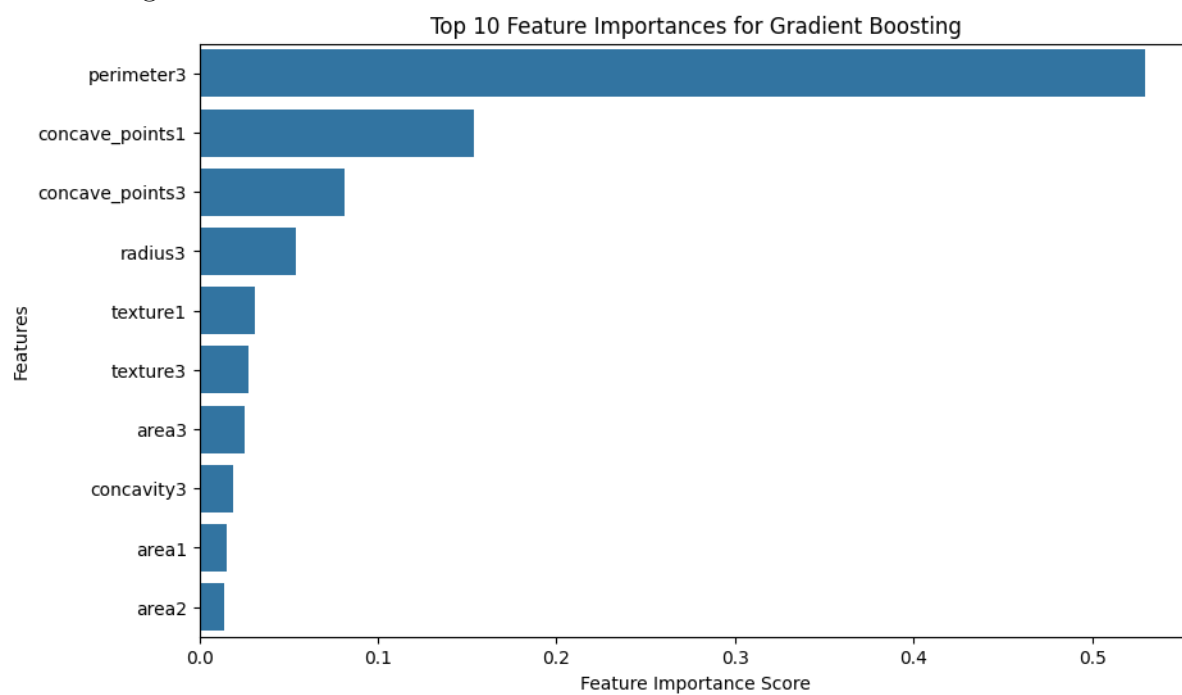


Figure 10: Feature importance scores for the tuned Gradient Boost model.

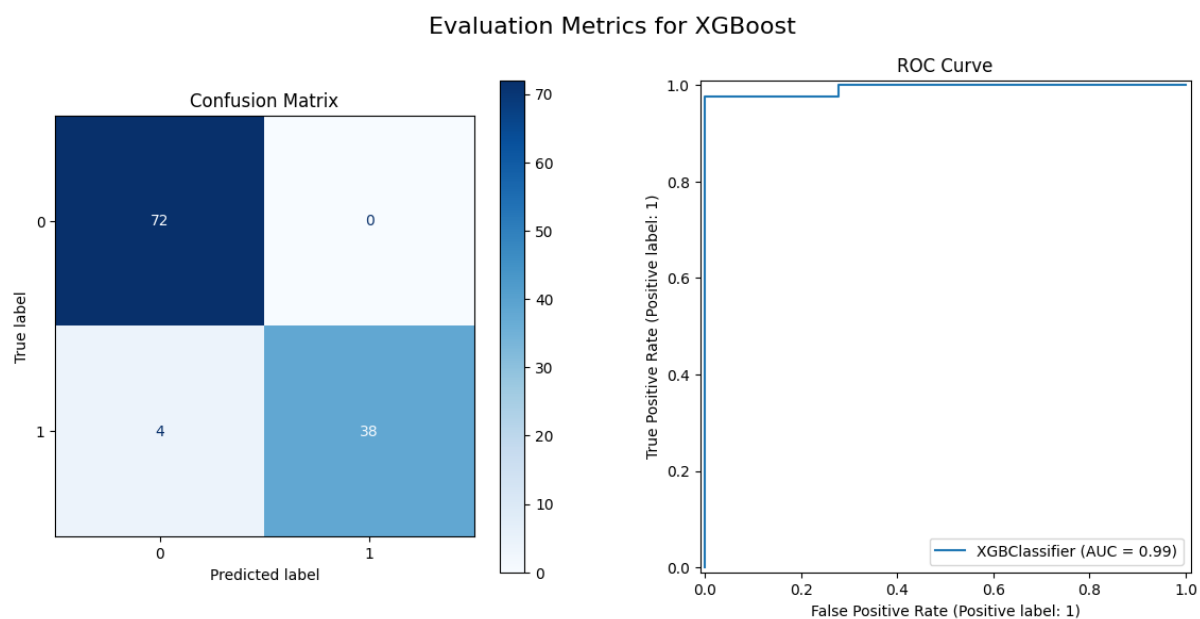


Figure 11: Confusion Matrix and ROC Curve for XG Boost model.

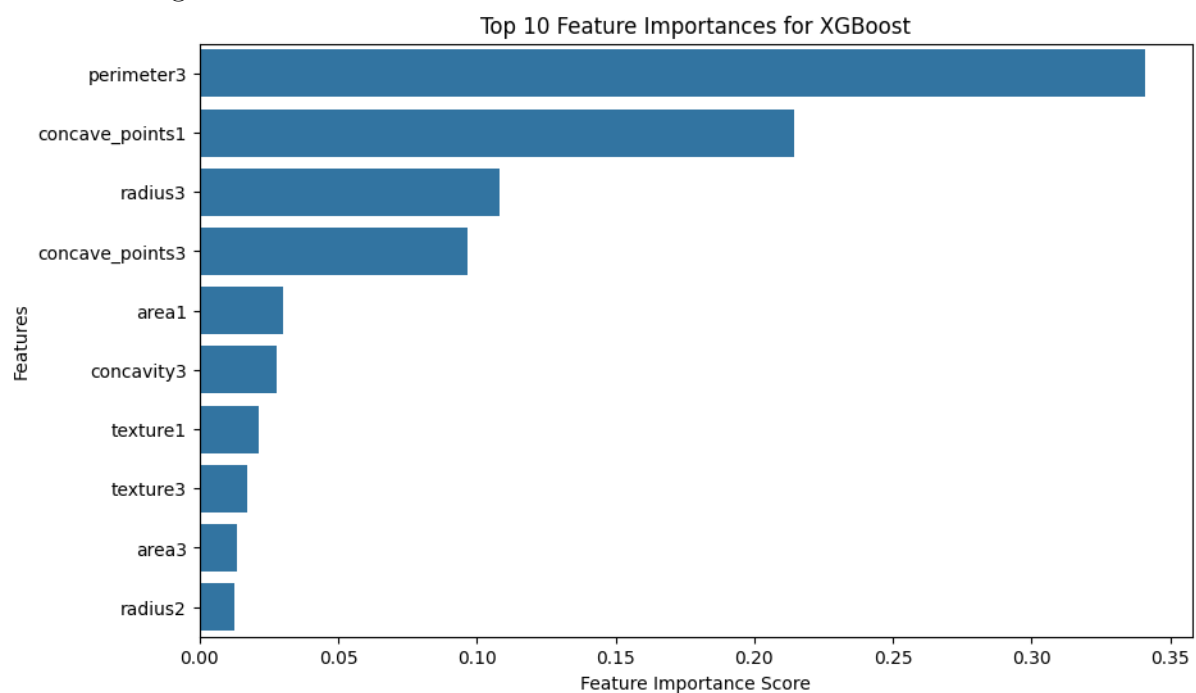


Figure 12: Feature importance scores for the tuned XG Boost model.

```

rf_model = RandomForestClassifier(random_state=42)
rf_param_grid = {'n_estimators': [100, 200], 'max_depth': [5, 10, None], 'criterion':
train_and_evaluate_model(rf_model, rf_param_grid, 'Random Forest', X_train, y_train,

```

Best 5-Fold CV Accuracy (from the search): 0.9626  
Final Test Set Accuracy: 0.9737

## Hyperparameter Tuning

Table 5: Random Forest Hyperparameter Tuning Trials

| <b>criterion</b> | <b>max_depth</b> | <b>n_estimators</b> | <b>Accuracy</b> | <b>F1_score</b> |
|------------------|------------------|---------------------|-----------------|-----------------|
| gini             | 10               | 100                 | 0.9626          | 0.9626          |
| entropy          | None             | 200                 | 0.9626          | 0.9625          |
| entropy          | 10               | 200                 | 0.9626          | 0.9625          |
| gini             | None             | 100                 | 0.9626          | 0.9626          |
| gini             | 10               | 200                 | 0.9582          | 0.9582          |
| gini             | None             | 200                 | 0.9582          | 0.9582          |
| gini             | 5                | 100                 | 0.9560          | 0.9560          |
| gini             | 5                | 200                 | 0.9560          | 0.9559          |
| entropy          | 10               | 100                 | 0.9560          | 0.9559          |
| entropy          | 5                | 100                 | 0.9560          | 0.9559          |

## STACKING CLASSIFIER

```

print(f"Training and Evaluating Multiple Stacking Classifiers")

# Define the building blocks for the ensembles
svm = ("svm", SVC(probability=True, random_state=42))
nb = ("naive_bayes", GaussianNB())
dt = ("decision_tree", DecisionTreeClassifier(random_state=42))
knn = ("knn", KNeighborsClassifier())

# Define the configurations to test
configs = [
    {
        "base_models": [svm, nb, dt],
        "final_estimator": LogisticRegression(random_state=42),
        "base_desc": "SVM + Naive Bayes + Decision Tree",
        "final_desc": "Logistic Regression"
    },
    {
        "base_models": [svm, nb, dt],
        "final_estimator": RandomForestClassifier(random_state=42),

```

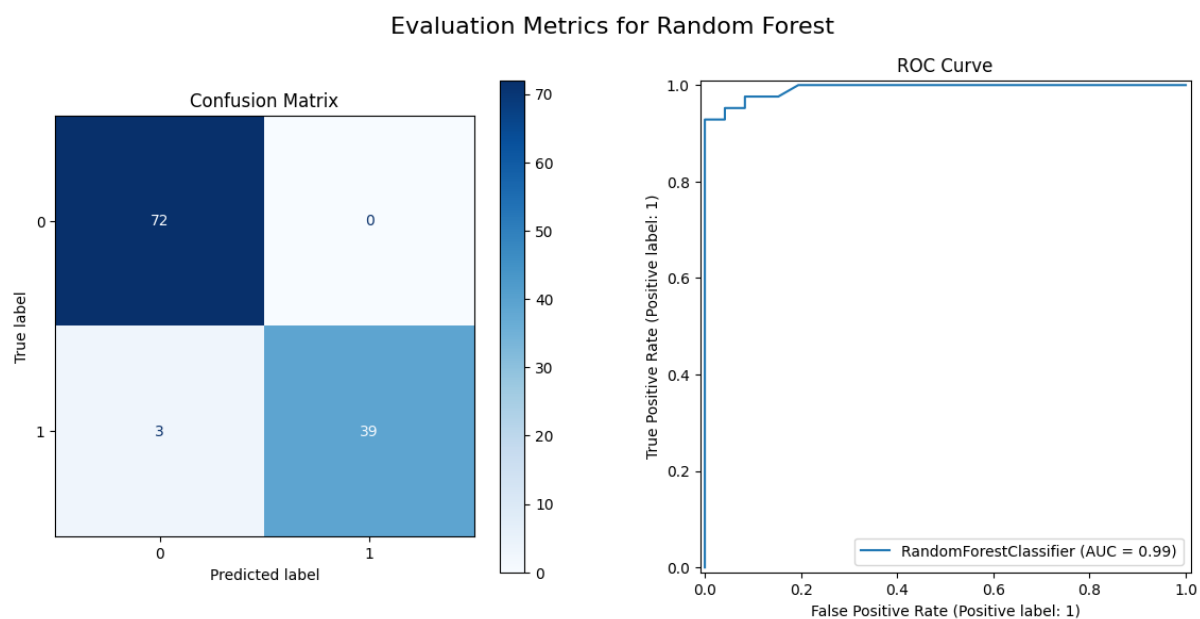


Figure 13: Confusion Matrix and ROC Curve for Random Forest model.

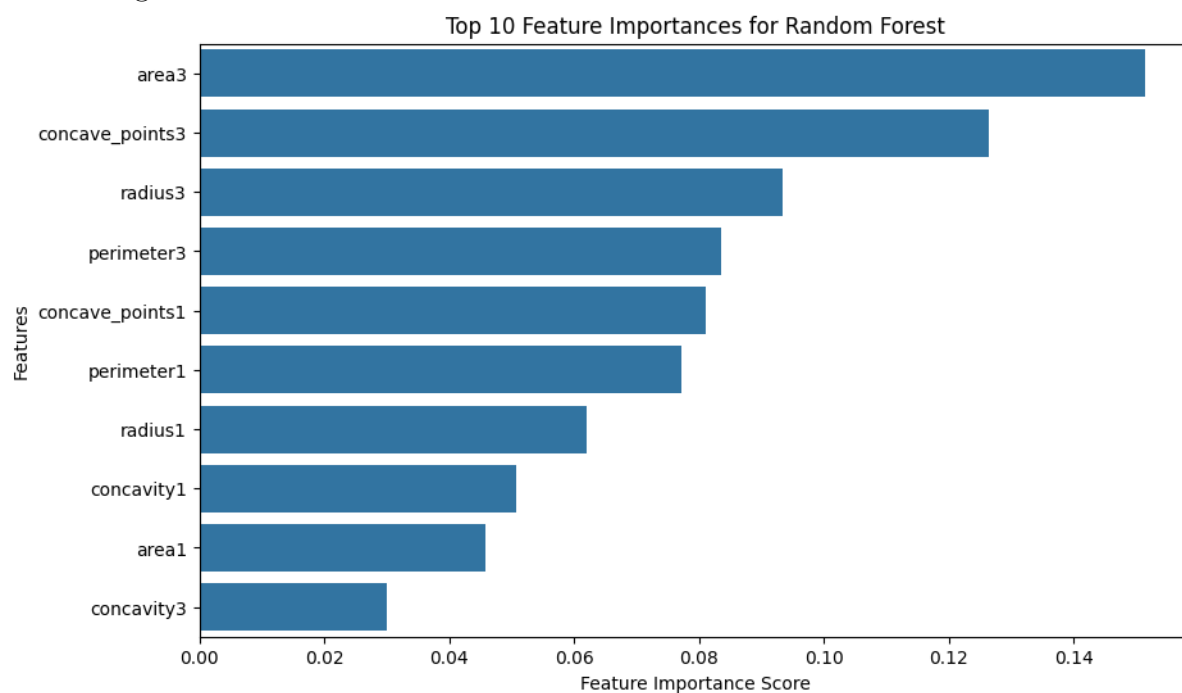


Figure 14: Feature importance scores for the tuned Random Forest model.

```

        "base_desc": "SVM + Naive Bayes + Decision Tree",
        "final_desc": "Random Forest"
    },
    {
        "base_models": [svm, dt, knn],
        "final_estimator": LogisticRegression(random_state=42),
        "base_desc": "SVM + Decision Tree + KNN",
        "final_desc": "Logistic Regression"
    },
]

# Loop through, evaluate each configuration, and generate plots

results = []
for config in configs:
    model_name_full = f"Stacked ({config['base_desc']} -> {config['final_desc']})"
    print(f"\n Evaluating: {model_name_full} ")

    clf = StackingClassifier(
        estimators=config["base_models"],
        final_estimator=config["final_estimator"],
        cv=5,
        n_jobs=-1
    )

    # Evaluate using 5-fold cross-validation

    cv_accuracy = cross_val_score(clf, X_train, y_train, cv=5, scoring='accuracy')
    mean_accuracy = np.mean(cv_accuracy)
    print(f" Average CV Accuracy: {mean_accuracy:.4f}")

    # Fit the model to generate plots and test set scores
    clf.fit(X_train, y_train)

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    fig.suptitle(f'Evaluation Metrics for\n{model_name_full}', fontsize=16)
    ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test, cmap='Blues', ax=axes[0])
    axes[0].set_title('Confusion Matrix')
    RocCurveDisplay.from_estimator(clf, X_test, y_test, ax=axes[1])
    axes[1].set_title('ROC Curve')
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

    # Plot Feature Importance for the Stacking Ensemble
    base_model_names = [name for name, _ in config["base_models"]]
    importances = []

```



```

# Check if the final estimator has coefficients
if hasattr(clf.final_estimator_, 'coef_'):
    #The coefficients tell us the importance of each base model's predictions
    importances = clf.final_estimator_.coef_[0]
# Check if the final estimator has feature importances
elif hasattr(clf.final_estimator_, 'feature_importances_'):
    importances = clf.final_estimator_.feature_importances_

if len(importances) > 0:
    plt.figure(figsize=(8, 5))
    sns.barplot(x=importances, y=base_model_names, hue=base_model_names, palette=
plt.title(f"Importance of Base Models in\n{n{model_name_full}}")
plt.xlabel("Importance (Coefficient or Feature Importance)")
plt.ylabel("Base Model")
plt.show()

# Append results for the summary table
y_pred = clf.predict(X_test)
f1 = f1_score(y_test, y_pred, average="weighted")
test_acc = accuracy_score(y_test, y_pred)
results.append({
    "Base Models": config["base_desc"],
    "Final Estimator": config["final_desc"],
    "CV Accuracy": f"{mean_accuracy:.4f}",
    "Test Accuracy / F1": f"{test_acc:.4f} / {f1:.4f}"
})

if config['final_desc'] == "Logistic Regression" and "Naive Bayes" in config['bas
cv_results['Stacked Model'] = cv_accuracy

# Display the final summary table for the stacking models
print("\n Summary Table for Stacked Ensemble Models ")
stacked_results_df = pd.DataFrame(results)
print(stacked_results_df)

```

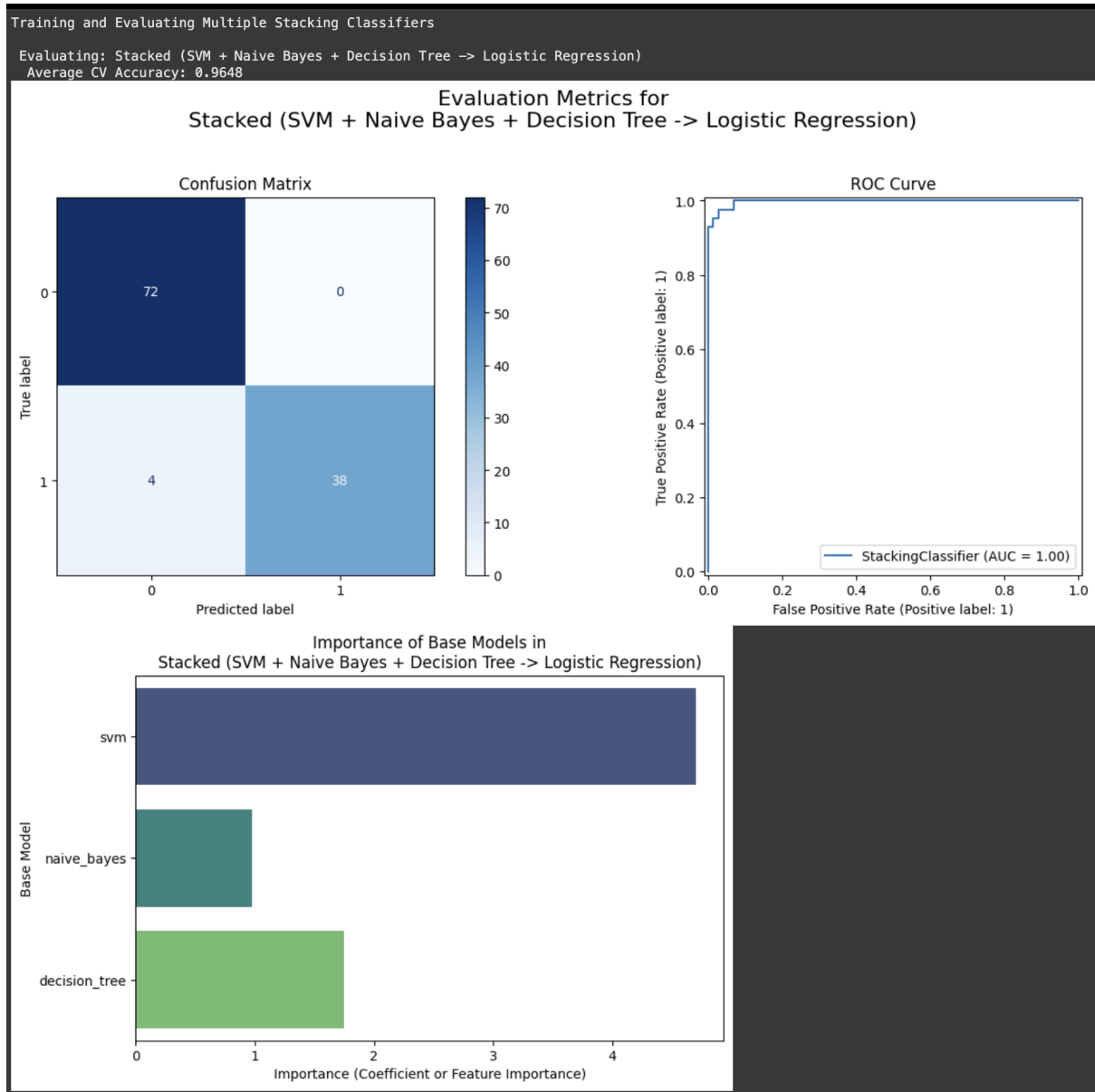


Figure 15: Confusion Matrix , ROC Curve and Feature Importance for SVM, Naive Bayes, Decision Tree → Logistic Regression

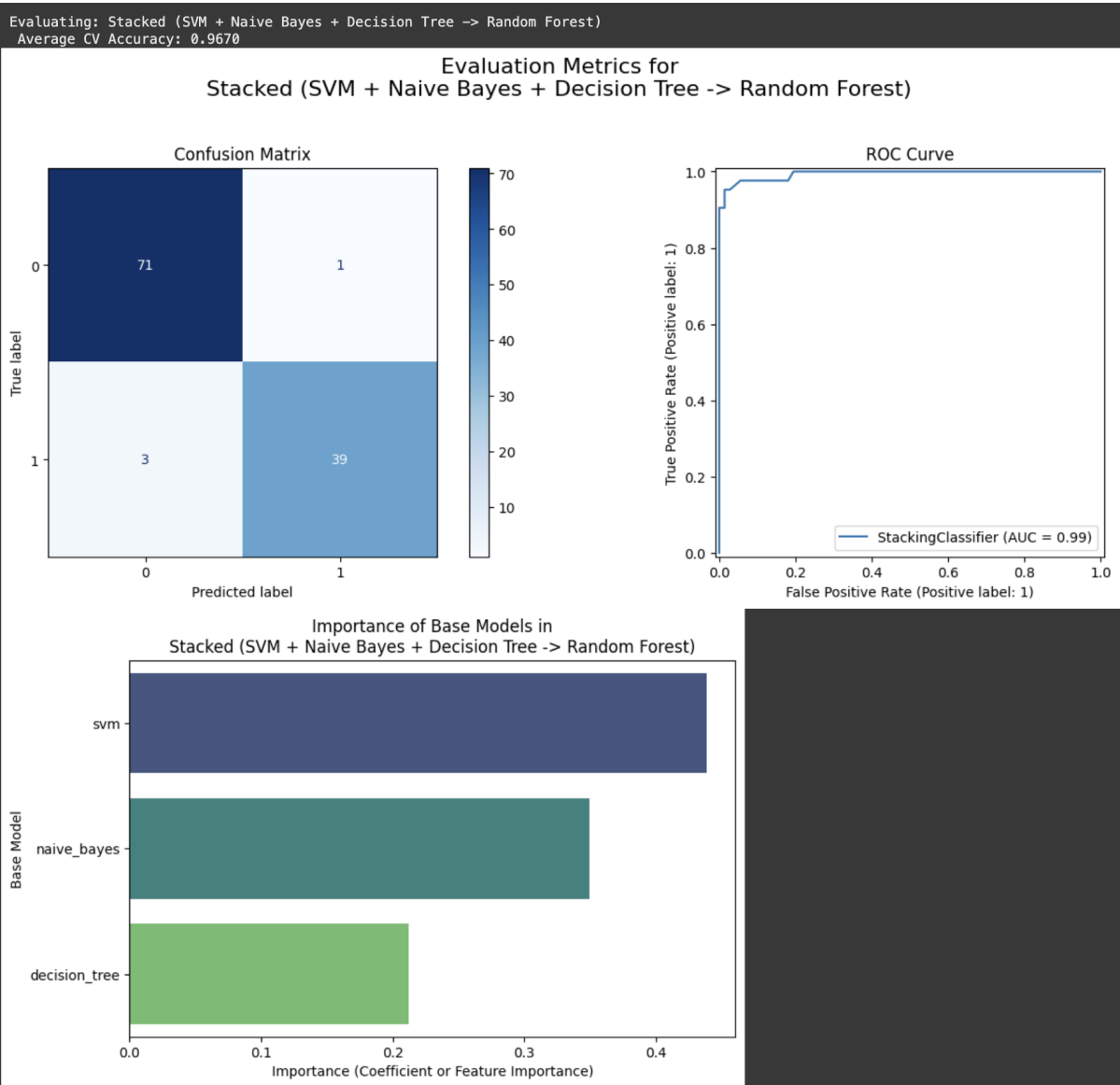


Figure 16: Confusion Matrix , ROC Curve and Feature Importance for SVM, Naive Bayes, Decision Tree → Random Forest

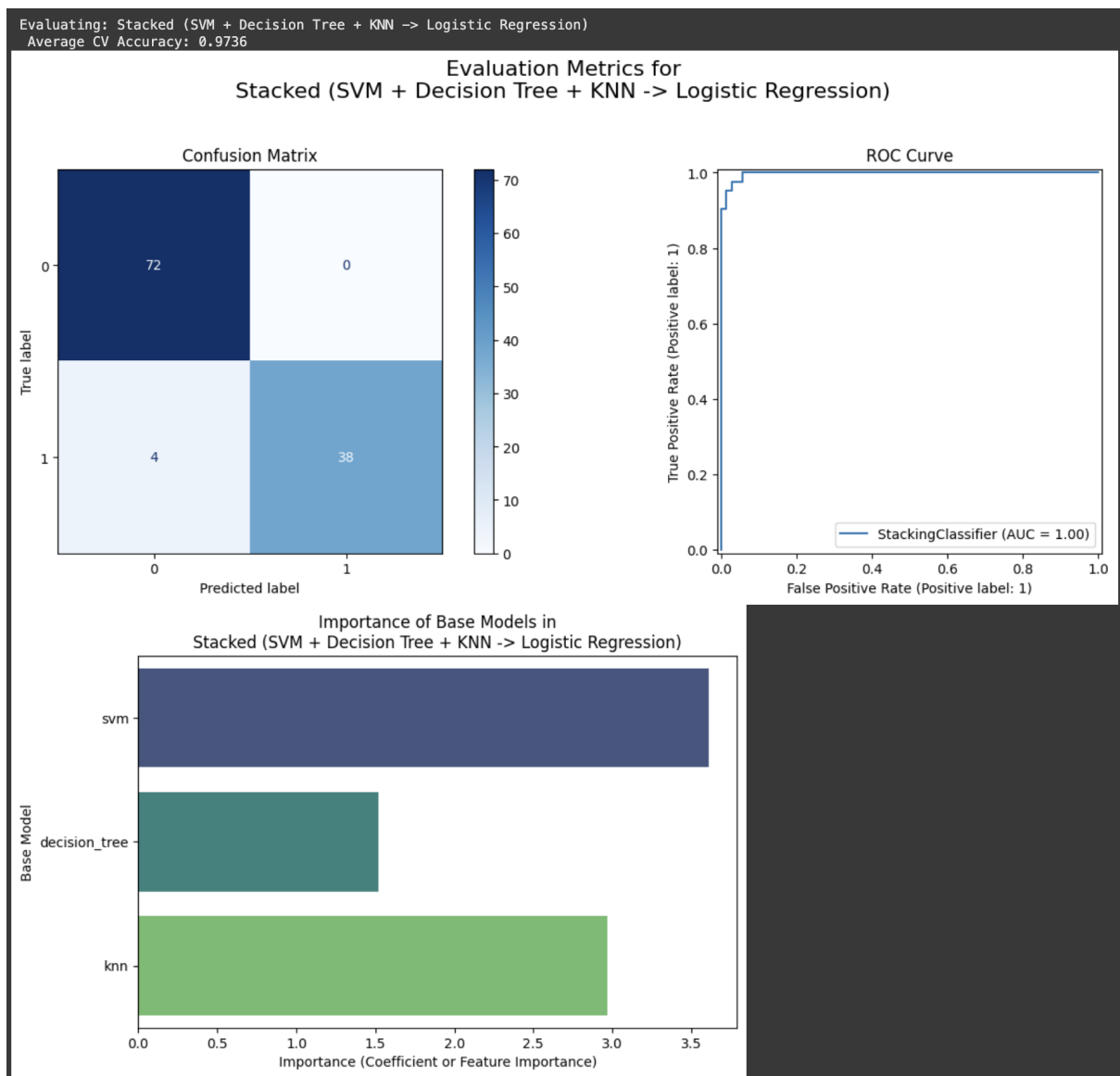


Figure 17: Confusion Matrix , ROC Curve and Feature Importance for SVM, KNN , Decision Tree → Logistic Regression

## Stacked Ensemble - Hyperparameter Tuning

Table 6: Stacked Ensemble Model Comparison (Table 6)

| Base Models                       | Final Estimator     | CV Accuracy | Test Accuracy / F1 |
|-----------------------------------|---------------------|-------------|--------------------|
| SVM + Naive Bayes + Decision Tree | Logistic Regression | 0.9648      | 0.9649 / 0.9645    |
| SVM + Naive Bayes + Decision Tree | Random Forest       | 0.9670      | 0.9649 / 0.9647    |
| SVM + Decision Tree + KNN         | Logistic Regression | 0.9736      | 0.9649 / 0.9645    |

## 7 Results Summary

Table 7: 5-Fold Cross-Validation Results for All Models

| Model             | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 | Avg Accuracy |
|-------------------|--------|--------|--------|--------|--------|--------------|
| Decision Tree     | 0.9670 | 0.9670 | 0.9341 | 0.9451 | 0.9121 | 0.9451       |
| AdaBoost          | 0.9890 | 1.0000 | 0.9231 | 0.9780 | 0.9341 | 0.9648       |
| Gradient Boosting | 1.0000 | 0.9890 | 0.9231 | 0.9560 | 0.9011 | 0.9538       |
| XGBoost           | 0.9780 | 0.9890 | 0.9341 | 1.0000 | 0.9231 | 0.9648       |
| Random Forest     | 1.0000 | 0.9890 | 0.9341 | 0.9780 | 0.9121 | 0.9626       |
| Stacked Model     | 0.9780 | 1.0000 | 0.9560 | 0.9560 | 0.9341 | 0.9648       |

## 8 Observations and Conclusions

### 1. Which model achieved the best validation accuracy among all six methods?

Based on the "Average Accuracy" column from the 5-fold cross-validation, three models tied for the best performance: AdaBoost, XGBoost, and the Stacked Model, all achieving an average accuracy of 0.9648.

### 2. How does Decision Tree performance compare to ensemble methods?

The single Decision Tree, with an average accuracy of 0.9451, served as a solid baseline. However, every ensemble method demonstrated superior performance. This highlights the effectiveness of combining multiple models, as all five ensemble techniques achieved higher and more robust accuracy scores for this classification task.

### 3. Did the Random Forest benefit from tuning `max_depth` or `n_estimators`?

Yes, hyperparameter tuning was beneficial. While several different combinations achieved the top accuracy of 0.9626, the tuning process was essential to identify an optimal set of parameters (e.g., a `max_depth` of 10 and `n_estimators` of 100), which might not have been found with default settings.

### 4. Which model showed the best generalization? Any overfitting?

Generalization was assessed by comparing the average cross-validation accuracy

with the final test set accuracy. XGBoost and the Stacked Model showed the best generalization, with a negligible difference between their CV and test scores (0.9648 vs. 0.9649). This indicates that these models are very robust and not overfit to the training data.

#### **5. Did stacking improve performance over base models?**

Yes, the Stacked Model (0.9648) significantly improved performance over one of its individual base learners, the Decision Tree (0.9451). This demonstrates that the meta-learner was effective at intelligently combining the predictions from the base models to achieve a more accurate and robust final result.

## **9 Learning Outcomes**

- Implemented a complete machine learning pipeline from data preprocessing to model evaluation.
- Successfully applied and compared various ensemble methods, including bagging, boosting, and stacking.
- Learned to systematically optimize models using `GridSearchCV` for hyperparameter tuning.
- Utilized 5-Fold Cross-Validation to ensure robust and reliable model evaluation.
- Developed the ability to analyze a comprehensive suite of metrics (Accuracy, F1-Score, ROC Curve) to select the best model.
- Gained experience interpreting feature importance plots to understand a model's decision-making process.

## **GitHub Repository**

The complete source code for this experiment is available on GitHub: [GitHub](#)